BATALLA NAVAL

Programación en red

8 DE SEPTIEMBRE DE 2026

YAZMIN RODRIGUEZ SALAZAR

JESÚS ONEFRE RODRIGUEZ CORTES

1. Introducción

En el ámbito de la programación de red, la comunicación entre aplicaciones distribuidas requiere el uso de protocolos de la capa de transporte. Los dos protocolos predominantes son TCP (Transmission Control Protocol) y UDP (User Datagram Protocol).

TCP es un protocolo orientado a conexión que garantiza la entrega ordenada y sin errores de los paquetes, estableciendo un enlace formal mediante un saludo de tres vías (three-way handshake) antes de iniciar la transferencia de datos. En contraste, UDP es un protocolo sin conexión que prioriza la velocidad y eficiencia sobre la fiabilidad, enviando datagramas sin asegurar su recepción o el orden de llegada.

El objetivo de esta actividad es implementar una arquitectura cliente-servidor utilizando la librería socket en Python para contrastar empíricamente las características de TCP y UDP mediante la simulación de un juego de Batalla Naval.

2. Desarrollo y Explicación del Código

2.1 Configuración de Sockets

Para establecer la comunicación en Python, se hizo uso del módulo socket. Dependiendo del

protocolo, la inicialización estructural varía:

- Para TCP: Se utiliza el parámetro socket.SOCK_STREAM para abrir un flujo continuo y confiable de datos.
- Para UDP: Se utiliza socket.SOCK_DGRAM para enviar mensajes individuales en forma de datagramas.

2.2 Implementación del Protocolo TCP

En el script servidor_TCP.py , el servidor se enlaza a una dirección IPv4 local y un puerto específico mediante la función bind() , poniéndose en modo de escucha con listen() .

Cuando un cliente solicita conectarse, el servidor acepta la conexión usando accept() , lo que genera un nuevo socket dedicado exclusivamente a gestionar la transferencia con ese cliente en particular. Por su parte, el script cliente_TCP.py utiliza el método connect() para establecer el handshake inicial y comenzar el intercambio de coordenadas del juego.

2.3 Implementación del Protocolo UDP

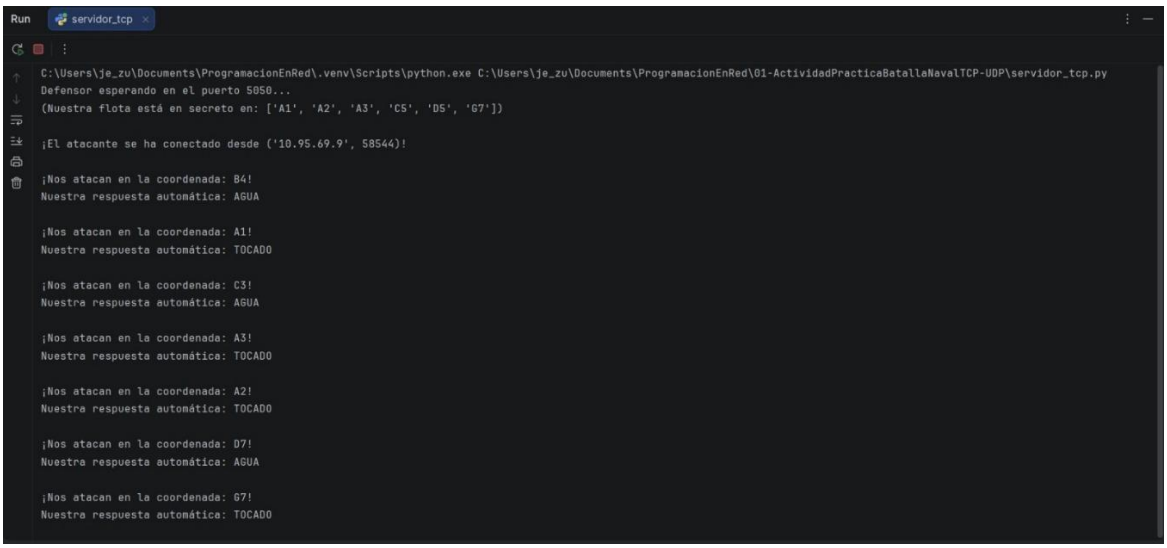
A diferencia de TCP, en el script servidor_UDP.py no se establecen conexiones persistentes ni se utiliza el método de escucha. El servidor simplemente ejecuta bind() y queda a la espera pasiva de recibir datagramas con recvfrom() , método que además de los datos, extrae la dirección de origen para poder responder. El script Cliente_UDP.py emite directamente los mensajes al puerto del servidor usando sendto() , sin validación previa del estado del servidor.

3. Evidencia de Ejecución

A continuación, se presentan las evidencias de la terminal mostrando la ejecución simultánea de los scripts. En las imágenes se comprueba el correcto flujo de intercambio de datos (envío de coordenadas de la batalla naval y confirmación de impacto) entre el cliente y el servidor.

TCP

Servidor



Cliente

```
/Universidad/Programacion_red/01-Actividad_Practica_TCP_UDP/cliente_TCP.py
¡Conectado a la base del Defensor! Prepara tu ataque.

Ingresa tu ataque (ej. B4): b4
Reporte de daños: AGUA

Ingresa tu ataque (ej. B4): A1
Reporte de daños: TOCADO

Ingresa tu ataque (ej. B4): C3
Reporte de daños: AGUA

Ingresa tu ataque (ej. B4): A3
Reporte de daños: TOCADO

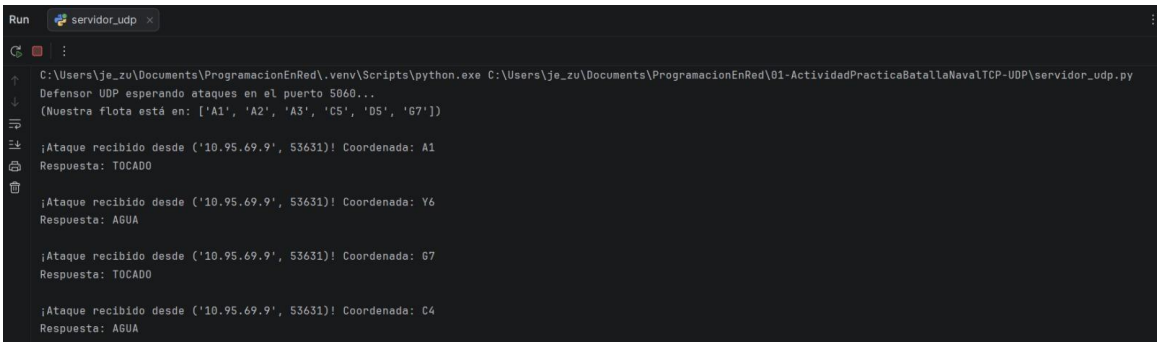
Ingresa tu ataque (ej. B4): A2
Reporte de daños: TOCADO

Ingresa tu ataque (ej. B4): D7
Reporte de daños: AGUA

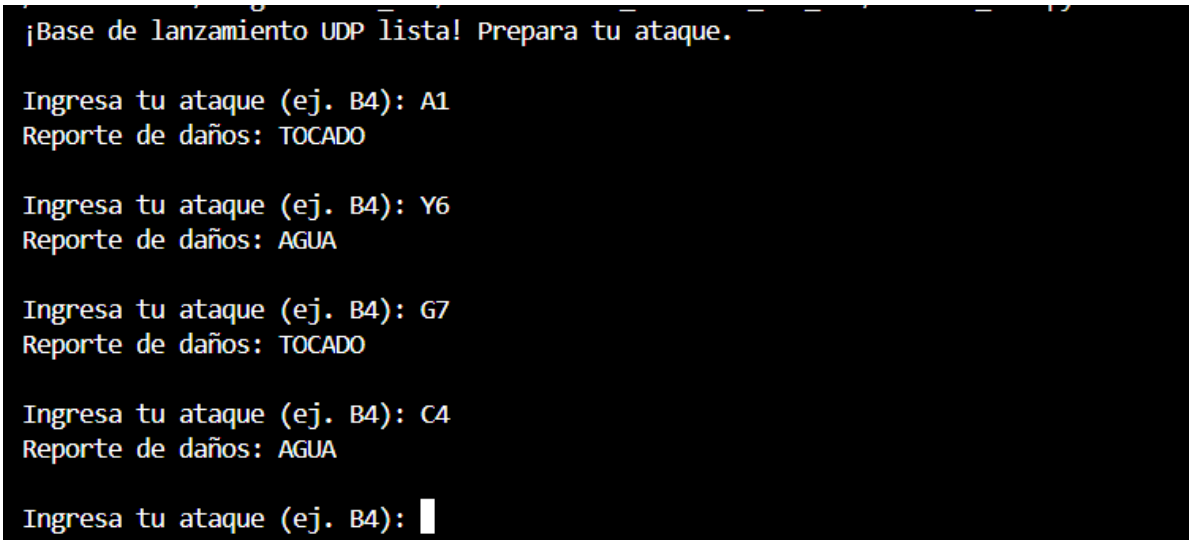
Ingresa tu ataque (ej. B4): G7
Reporte de daños: TOCADO
```

UDP

Servidor



Cliente



4. Conclusión

Mediante la codificación y ejecución de estos scripts, se comprobó operativamente la diferencia teórica entre TCP y UDP. La programación del juego en TCP requiere un manejo más estructurado del ciclo de vida de la conexión, pero es idóneo para una Batalla Naval, ya que la pérdida de un solo paquete desfasaría los turnos y arruinaría el estado de la partida. Por otro lado, la programación en UDP resultó ser más directa y requiere menos líneas de código de preparación; sin embargo, en un entorno de red real, depender de UDP requeriría programar mecanismos manuales de retransmisión en la capa de aplicación para garantizar que los "ataques" lleguen a su destino.